TD VHDL 2/3 – Électronique Numérique

Circuits séquentiels et Automates

Notes et Conseils

Remarque : Ce TD constitue la suite de l'introduction à la description matérielle en VHDL. Il aborde les circuits séquentiels (compteurs) et la description d'automates aux niveaux logique et RTL. Il est conseillé de tester les codes sur un simulateur (GHDL) pour valider vos acquis.

1 Descriptions de chemin de données séquentiel

Questions

1. Coder en VHDL au moins deux architectures numériques qui réalisent un compteur, qui compte de 0 à 255, en respectant l'entité donnée ci-dessous. On note que les opérandes sont codés en **unsigned(7 downto 0)** (octets non-signés). On prendra soin de dessiner le circuit avant de coder l'architecture.
2. Tester le circuit à l'aide du testbench fourni.

Solution

On présente 2 architectures, *in fine* identiques, mais codées différemment. L'architecture 2 semble plus naturelle ; l'architecture 1 sépare explicitement partie combinatoire et partie séquentielle.

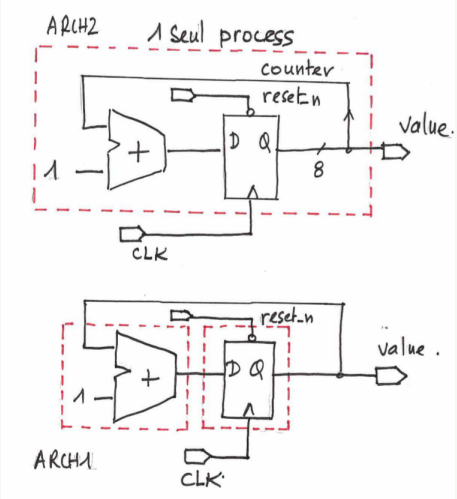


FIGURE 1 – Architecture du compteur, deux organisations de code possibles : soit 1 seul processus, soit 2 processus (1 processus + 1 assignation concurrente)

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity compteur is
  port(
    reset_n : in std_logic;
    clk : in std_logic;
    value : out unsigned(7 downto 0)
  );
end compteur;

architecture arch1 of compteur is
  signal counter : unsigned(7 downto 0);
  signal counter_comb : unsigned(7 downto 0);
begin

  counter_p : process(reset_n,clk)
  begin
    if reset_n='0' then
      counter <= "00000000";
    elsif rising_edge(clk) then
      counter <= counter_comb;
    end if;
  end process;

  counter_comb <= counter + 1;

  value <= counter;
end arch1;

architecture arch2 of compteur is
  signal counter : unsigned(7 downto 0);
begin
  counter_p : process(reset_n,clk)
  begin
    if reset_n='0' then
      counter <= "00000000";
    elsif rising_edge(clk) then
```

2 Descriptions d'automates au niveau logique

Soit un automate bien connu des informaticiens parisiens, sur la figure **??**.

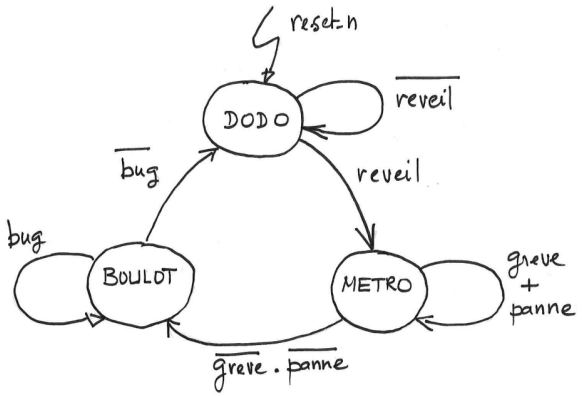


FIGURE 2 – Diagramme états-transitions de l'automate "I love Paris"

Questions

1. Vérifier que, dans le cas d'un encodage one-hot, les équations d'état suivant sont bien données par :

$$\begin{cases} D_0 &= Q_0.\overline{reveil} + Q_2.\overline{bug} \\ D_1 &= Q_0.reveil + Q_1.(greve + panne) \\ D_2 &= Q_1.\overline{greve}.\overline{panne} + Q_2.bug \end{cases}$$

2. Coder l'automate "I_love_Paris" en VHDL, au niveau logique, en prenant soin de bien coder les bascules D à l'aide de processus. Respectez l'entité fournie ici. On utilisera une assignation conditionnelle (when) pour coder le signal de sortie "up_and_running".

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity i_love_paris is
  port(
    reset_n : in std_logic;
    clk : in std_logic;
    reveil : in std_logic;
    panne : in std_logic;
    greve : in std_logic;
    bug : in std_logic;
    up_and_running : out std_logic
  );
end i_love_paris;
```

3. Utilisez le testbench donné sous Moodle pour tester votre automate. Observez le résultat.

Solution

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity i_love_paris is
  port(
    reset_n : in std_logic;
    clk : in std_logic;
    reveil : in std_logic;
    panne : in std_logic;
    greve : in std_logic;
    bug : in std_logic;
    up_and_running : out std_logic
  );
end i_love_paris;

architecture logic of i_love_paris is
  signal state, next_state : std_logic_vector(2 downto 0);
  --preferable to :
  --signal d, q : std_logic_vector(2 downto 0);
begin
  process(reset_n,clk)
  begin
    if reset_n='0' then
      state <= "001";--dodo
    elsif rising_edge(clk) then
      state <= next_state;
    end if;
  end process;

  next_state(0) <= (state(0) and not(reveil)) or (state(2) and not(bug));
  next_state(1) <= (state(0) and reveil) or (state(1) and (greve or panne));
  next_state(2) <= (state(2) and bug) or (state(1) and (not(greve) and not(panne)
      );

end logic;
```

3 Descriptions d'automates au niveau RTL

Ce niveau d'abstraction RTL est le plus couramment utilisé, car il permet de s'affranchir des détails des équations logiques, et décrire des automates (et *micro-architectures*) de manière plus naturelle.

Le niveau RTL repose sur la notion d'*inférence matérielle* : l'Électronicien doit connaître certains *motifs de conception*[1], afin de permettre à l'outil de synthèse d'établir automatiquement les équations logiques sous-jacentes.

Le cas des automates d'états finis est instructif en ce sens. Voici un exemple de codage VHDL, qui décrit un automate, sans expliciter les équations logiques sous-jacentes, ni l'encodage des états. Pour information, le synthétiseur peut choisir de lui-même un encodage qui maximise la fréquence de fonctionnement du circuit (encodage one-hot généralement), ou tout autre type de contraintes imposées par le concepteur.

1. "Design patterns", en anglais.

Exemple d'automate de niveau RTL

Nous donnons ici à titre d'exemple un diagramme états-transitions, ainsi que son code VHDL de niveau RTL. Notez que nous avons séparé le processus ici appelé "next_state_p", qui code la fonction de transition (calcul de l'état suivant), et la sortie de l'automate.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity automate is
  port(
    reset_n : in std_logic;
    clk : in std_logic;
    evenement : in std_logic;
    sortie : out std_logic
  );
end automate;

architecture rtl of automate is

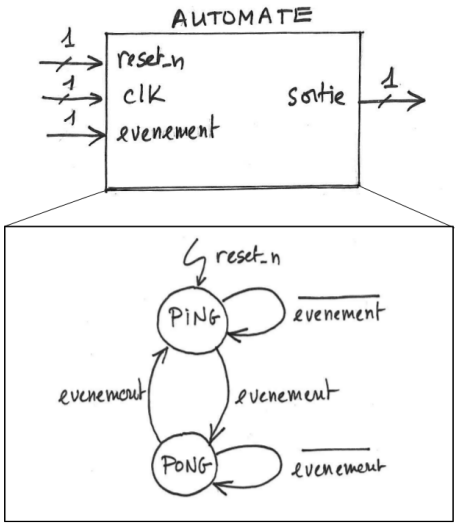
  type state_type is (PING,PONG);
  signal state, next_state : state_type;

begin
  process(reset_n,clk)
  begin
    if reset_n='0' then
      state <= PING;
    elsif rising_edge(clk) then
      state <= next_state;
    end if;
  end process;

  -- logique d'etat suivant
  next_state_p: process(state,evenement)
  begin
    next_state <= state;--default
    case state is
      when PING =>
       if evenement='1' then
         next_state <= PONG;
       end if;
      when PONG =>
       if evenement='1' then
         next_state <= PING;
       end if;
      when others =>
        null;
    end case;
  end process;

  -- logique de sortie
  sortie <='1' when state=PONG else '0';

end rtl;
```



Questions

1. Prenez le temps de comprendre le codage utilisé ici.
2. Coder l'automate "I_love_Paris" en VHDL, au niveau RTL, en vous inspirant de l'exemple donné.
3. Utilisez le même testbench que précédemment pour tester votre automate. Modifier uniquement le nom de l'architecture associée à l'entité instanciée.

Solution

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity i_love_paris is
  port(
    reset_n : in std_logic;
    clk : in std_logic;
    reveil : in std_logic;
    panne : in std_logic;
    greve : in std_logic;
    bug : in std_logic;
    up_and_running : out std_logic
  );
end i_love_paris;

architecture rtl of i_love_paris is

  type state_type is (DODO,METRO,BOULOT);
  signal state, next_state : state_type;

begin
  process(reset_n,clk)
  begin
    if reset_n='0' then
      state <= DODO;
    elsif rising_edge(clk) then
      state <= next_state;
    end if;
  end process;

  next_state_p: process(state,reveil,greve,panne,bug)
  begin
    next_state <= state;--default
    case state is
      when DODO =>
       if reveil='1' then
         next_state <= METRO;
       end if;
      when METRO =>
       if greve='0' and panne='0' then
         next_state <= BOULOT;
       end if;
      when BOULOT =>
        if bug='0' then
          next_state <= DODO;
        end if;
      when others =>
        null;
    end case;
  end process;

  -- output logic :
  up_and_running <= '0' when state=DODO else '1';

end rtl;
```

